Voices in the Comments:
CS621 Project

Comparing audience sentiment and engagement for two gaming influencers — **videogamedunkey** and **Game Grumps**

Data Analytics Project Report | July 2026 | figures show actual collected data

1. Introduction

YouTube comment sections are one of the most direct, unfiltered records of how an audience feels about a creator. This project applies the standard data science pipeline — **data source → ingestion → engineering → analytics → visualization → delivery** — to the comment sections of two gaming channels of similar scale but very different formats: **videogamedunkey** ("dunkey", 7.57M subscribers with tightly edited comedy game reviews) and **Game Grumps** (5.45M subscribers and a long-running let's-play duo whose hosts are the show).

The main question is; do a review channel and a personality channel earn different kinds of audience conversation in sentiment, timing, and vocabulary? Using the YouTube Data API v3, I collected **20,080 top comments** from each channel's 10 most-viewed videos (~10,000 per channel), cleaned them to **3,006 high-quality rows**, and ran basic descriptive analytics plus five advanced analyses: polarity/sentiment, time series, spatial proxy, emoji emotion, and head-to-head comparison.

The intended customers are the creators and their channel managers, who can use the findings to understand audience reception, and brand/sponsorship analysts deciding which community fits a campaign.

2. Data Source

Data comes from the **YouTube Data API v3**). For each channel we page the full uploads playlist and rank every video by its actual view count. The API's own "order by views" search proved unreliable on large channels. Then page through each top video's *commentThreads* ordered by **relevance**, collecting the highly-liked comments that define its conversation. Every raw response is persisted to **JSON files** (one per video) before any processing, so the pipeline re-runs without re-spending quota. In memory, data moves through Python **lists and dictionaries** during collection, then **pandas Series and DataFrames** for cleaning and analysis. This is a sample raw record (one comment as returned by commentThreads, simplified):


```
{
  "comment_id": "UgzKp3xQ9rLmW4aVn8B4AaABAg",
  "video_id": "lN-rVlMIJZs",
  "influencer": "dunkey",
  "author": "@portalfan_2015",
  "text": "What kind of GLaDOS level super computer do you have to
           process that fight at the end!!??",
  "likes": 25654,
  "published_at": "2015-03-12T20:03:11Z"
}
```


Sample of collected comments (after loading into a pandas DataFrame):

influencer	author	text	likes	published_at
dunkey	@portalfan_2015	What kind of GLaDOS level super computer do you have to process that fight at the end!!??	25,654	2015-03-12
dunkey	@returning_fan	Man I still come back to this video now and then. A classic	18,418	2017-04-11
Game Grumps	@grumpfan_x	the best, laughed so damn hard when I first saw this.	31,039	2016-03-10
Game Grumps	@skitfan	I love the swapping between the skit and reality	8,377	2018-05-01

3. The Data Science Pipeline, Stage by Stage

3.1 Data Ingestion

- Rank each channel's full uploads catalog by real view counts and keep the top 10 videos per channel.
- Fetch each top video's comment threads by relevance, up to 1,000 comments per video (10 API calls), so both channels contribute ~10,000 raw comments.
- Persist every response as JSON plus a video-metadata file with upload dates.

Actual ingestion result:

Influencer	Catalog size	Videos fetched	Raw comments	JSON files
dunkey	995 videos	10	10,000	10
Game Grumps	9,881 videos	10	10,080	10
Total		20	20,080	20

3.2 Data Engineering

- Load raw JSON into lists of dictionaries, flatten into one pandas DataFrame (one row per comment) plus Series for per-field analysis.
- **Remove empty comments** — YouTube blocks empty posts, so this rule caught 0 of 20,080 rows.
- **Remove non-English comments** — langdetect, seeded so every run is reproducible; drops ~12% of the corpus.
- **Remove duplicates** — identical text on the same channel (copy-paste and bot spam).
- **Remove outdated comments** — a comment is outdated if posted more than 18 months after *its own video's upload*. This per-video “reception window” replaced a fixed calendar cutoff that had kept only 15% of one channel's comments because its biggest videos are older.
- Derive columns (length, emoji list, weeks-since-upload, cleaned text) and save the cleaned dataset to CSV/JSON.

Actual cleaning funnel — 20,080 raw → 3,006 cleaned rows:

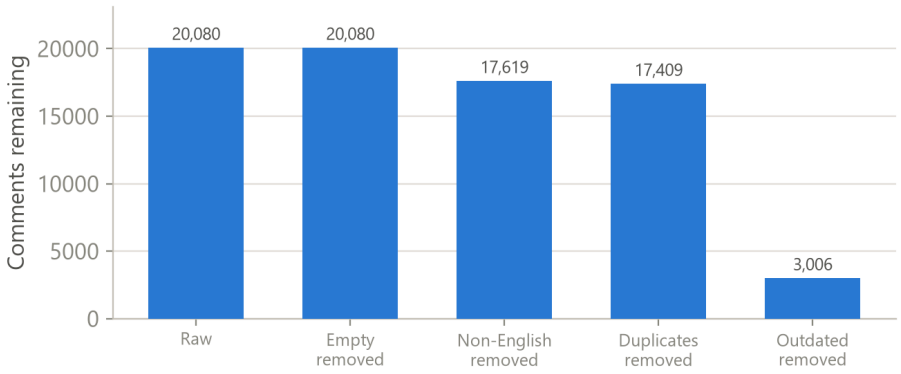

Figure 1. Cleaning funnel (actual data). The reception-window rule does most of the work.

3.3 Analytics Computation

Basic analytics:

Metric	dunkey	Game Grumps
Raw comments collected	10,000	10,080
Cleaned comments	1,262	1,744
Unique reviewers (authors)	1,223 (97%)	1,699 (97%)

Mean comment length (chars)	102	90
Median likes per comment	120	19
Comments containing emoji	2%	2%

Table: basic-analytics summary (actual data).

Advanced analytics:

- **Polarity / sentiment** — VADER scores every cleaned comment. Game Grumps runs warmer: 49.3% positive / 22.7% negative vs dunkey's 46.1% / 27.8%.
- **Time series** — comments bucketed by weeks since their video's upload: 58% of dunkey's conversation lands in the first 4 weeks; a quarter of Game Grumps' conversation arrives 6+ months after upload.
- **Spatial proxy** — the API exposes no commenter location, so posting-hour distributions (peaks at 18–20 UTC) and the language mix of removed comments approximate audience geography
- **Emoji emotion** — emoji mapped to emotions; joy dominates both channels; Game Grumps shows 5× the love emoji.
- **Influencer comparison** — every metric side by side, testing the hypothesis that a review audience reacts to videos while a personality audience talks about the hosts.

Sentiment split per channel:

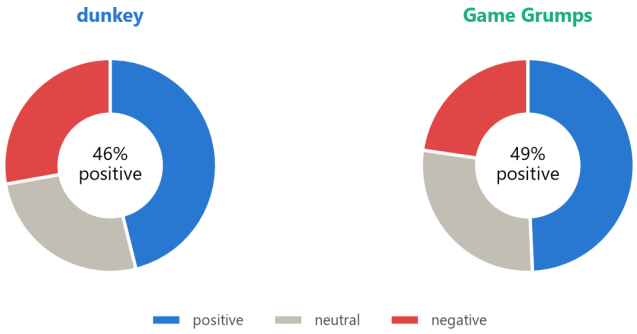

Figure 2. VADER sentiment, all 3,006 cleaned comments.

3.4 Visualization

All figures are produced with matplotlib inside the demo program using one consistent style: blue for dunkey, green for Game Grumps. Chart families: **bar charts** (cleaning funnel, emoji emotions), **pie charts** (sentiment), **boxplots** and **scatter plots** (lengths, likes), **time series** (reception curves), **spatial visualization** (posting-hour heatmap), and **word clouds**.

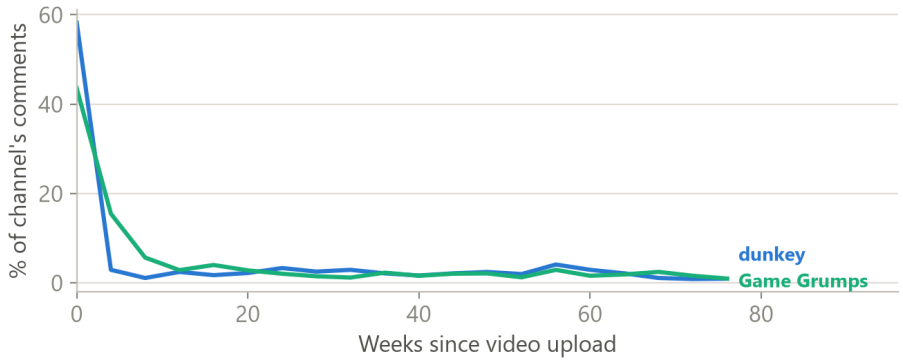

Figure 3. Reception curves (actual data): % of each channel's comments by weeks since video upload.

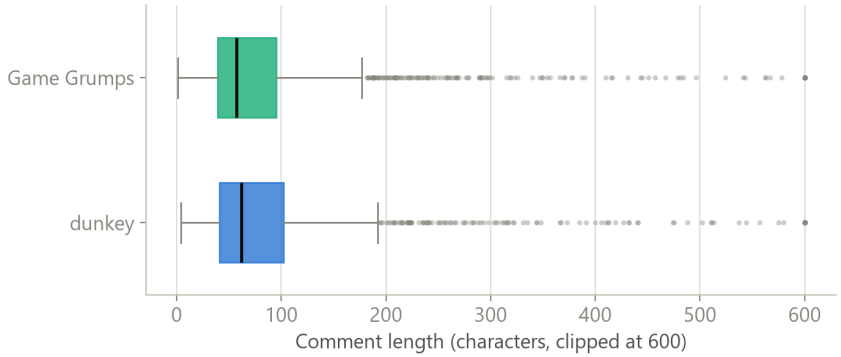

Figure 4. Comment lengths (actual data) — medians are one-liners on both channels.

Figure 5. Word clouds (actual data): dunkey's vocabulary is about games (dark souls, graphics, mario); Game Grumps' is about the hosts (arin, grumps, show).

3.5 Effort Summary

Stage	Main work	Hours
Data ingestion	API setup, catalog ranking, collector, JSON persistence	8
Data engineering	DataFrame build, four cleaning rules, derived columns	10
Analytics computation	Basic stats + five advanced analyses	14
Visualization	Seven chart families, consistent styling	8
Deliverables	Report, slides, demo program polish & documentation	8
Total		48

4. Deliverables

- **Raw dataset** — 20,080 comments as the original JSON files (20 files, one per video), exactly as returned by the API, plus video upload metadata.
- **Cleaned dataset** — 3,006 rows as CSV/JSON with derived columns and a data dictionary.
- **Written report** — this document: methodology, findings, and limitations.
- **Slide deck** — a 12-slide PowerPoint presentation of the headline findings.
- **Python program** — a documented ~330-line pipeline (ingestion → cleaning → analytics → charts) that reproduces every figure from the raw JSON.

Delivery is a single GitHub repository (code, datasets, README) plus the report and deck, followed by a live walkthrough of the program.

5. Professional Peer Feedback

Overall impression. A well-scoped, realistic project whose stages map cleanly onto the data science blueprint. Fixing the sample at each channel's ten most-viewed uploads keeps the comparison symmetric, and choosing two channels of similar scale but different formats gives the comparison a genuine hypothesis — which the vocabulary and timing analyses ended up supporting.

Nice ideas. Persisting raw JSON before any processing protects the day-limited quota and makes the pipeline reproducible. Ranking the uploads catalog by real view counts after discovering the search API's unreliability shows healthy distrust of convenient endpoints. Emoji emotion analysis is a clever answer to the weakness of text sentiment on short comments.

Challenges. (1) No commenter location exists in the API — the posting-hour/language proxy must stay clearly labeled. (2) VADER misreads gaming irony ("this game is disgusting" can be praise), so between-channel gaps matter more than absolute levels; a ~200-comment hand-labeled sample would quantify the error. (3) Language detection is unreliable on very short comments ('lol', 'GG'). (4) Relevance-ordered fetching samples each video's *top* comments, not a random sample. Findings describe the visible conversation, not every commenter.